

EDIYA_MEJANG

Supabase 레시피 관리 Web 서비스 인계서

별도 Web 관리자 서비스가 기존 앱의 레시피 데이터를 안전하게 조회·등록·수정·게시·삭제하기 위한 구현 계약

핵심 보안 경고

현재 모바일 앱의 anon 쓰기 정책은 공개 Web 서비스에 그대로 사용하면 안 됩니다.
Supabase Auth와 app_metadata 기반 관리자 RLS 전환을 권장합니다.

작성 기준일 2026-08-30 · Asia/Seoul

Supabase project ref: nbeupbzteeaeuznuaibm

목차

- 1. 핵심 요약
- 2. 2026-08-30 실데이터 읽기 점검
- 3. 현재 데이터 흐름
- 4. 연결 환경변수
- 5. 테이블 계약
- 6. 레시피 상세 JSON 계약
- 7. 초성(chosung) 생성 계약
- 8. 조회 구현 예시
- 9. 이미지 업로드 계약
- 10. 등록 및 수정
- 11. 게시 상태 관리
- 12. 영구 삭제 절차
- 13. 보안 모델
- 14. 현재 스키마의 개선 권고
- 15. Web 구현 체크리스트
- 16. 다른 AI에게 전달할 시작 프롬프트
- 17. 근거 파일과 공식 참고자료

작성 기준일: 2026-08-30 (Asia/Seoul)

대상: 현재 Expo 앱과 같은 Supabase 데이터를 조회하고, 별도 Web 관리 서비스에서 매장 레시피를 등록·수정·게시·삭제하려는 개발자 또는 AI 에이전트

1. 핵심 요약

- Supabase 프로젝트 ref: [nbeupbzteeaeuznuaibm](#)
- API URL: <https://nbeupbzteeaeuznuaibm.supabase.co>
- 레시피 마스터: [public.recipes](#)
- 레시피 상세: [public.recipe_details](#)
- 연관 통합검색: [public.find_entries](#), [public.find_entry_keywords](#)
- 이미지 버킷: [recipe-content](#) (현재 public bucket)
- 앱의 직원 화면은 Supabase를 직접 읽지 않고, 설정 화면에서 동기화한 버전형 기기 캐시를 사용한다.
- 직원 화면에 표시되는 데이터는 기본적으로 **status = 'published'**인 레시피다.
- 브라우저 코드에는 publishable key만 사용한다. [service_role](#) 또는 secret key를 절대 넣지 않는다.
- 현재 모바일 사장님 화면은 로컬 비밀번호 뒤에서 anon 쓰기를 허용하는 trusted-client 구조다. 인터넷에 공개될 Web 관리 서비스의 보안 모델로 그대로 복제하면 안 된다.

2. 2026-08-30 실데이터 읽기 점검

앱의 [.env.local](#)에 설정된 publishable/anon 연결로 읽기 전용 확인한 결과다.

- [recipes](#): 87행, 전부 **published**, 현재 카테고리는 모두 **음료**
- [recipe_details](#): 87행
- [find_entries](#): 2행 ([material](#) 1, [pos](#) 1)
- [find_entry_keywords](#): 4행
- 상세 JSON 배열 형식 오류: 0건
- 상세 JSON 안에 앱 런타임 전용 [image](#), [imageUri](#)가 저장된 사례: 0건
- 상세 visual: 14개 (Storage 경로가 있는 항목 13개, 텍스트 중심 항목 1개)
- [recipe-content](#) 버킷 목록 조회 가능
- 주의: POS 통합검색 항목 1개가 현재 존재하지 않는 recipe ID를 참조한다. 영구 삭제 기능을 만들기 전에 해당 항목을 재 연결하거나 archive해야 한다.

이 점검은 공개 Data API로 확인한 결과다. 실제 관리자용 GRANT, 모든 RLS 정책 정의, 트리거 및 함수 본문은 관리자 SQL 권한으로 별도 점검해야 한다.

3. 현재 데이터 흐름

1. 사장님 관리 화면 또는 새 Web 관리 서비스가 Supabase의 source row를 편집한다.
2. 이미지는 [recipe-content](#) Storage에 업로드하고 DB JSON에는 object path만 저장한다.
3. 레시피 저장 시 [recipes](#)와 [recipe_details](#)가 함께 갱신되어야 한다.
4. 앱의 설정 업데이트 기능은 **published** 레시피와 통합검색을 가져온다.
5. 참조된 이미지를 기기에 내려받고 새 캐시 generation을 검증한다.

6. 검증 성공 후에만 active pointer를 새 generation으로 교체한다.

7. 동기화 실패 시 이전 캐시는 유지된다.

따라서 Web에서 저장한 내용이 직원 화면에 즉시 직접 반영되는 구조는 아니다. DB 저장 후 앱에서 콘텐츠 업데이트 동작을 수행해야 새 레시피가 기기 캐시에 반영된다.

4. 연결 환경변수

Web 프로젝트에는 다음 이름을 권장한다.

```
NEXT_PUBLIC_SUPABASE_URL=https://nbeupbzteeaeuznuaibm.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<Supabase publishable key>
```

- 실제 키 값은 이 문서에 포함하지 않는다.
- 브라우저 번들에는 publishable key만 둔다.
- **SUPABASE_SERVICE_ROLE_KEY**가 필요하다면 서버 런타임의 secret으로만 보관하고 클라이언트 코드, 로그, PDF, Git에 넣지 않는다.
- 권장 구조는 Supabase Auth 로그인 후 **app_metadata.app_role = 'sajang'** 같은 서버 관리 claim을 RLS에서 확인하는 방식이다.
- **user_metadata**는 사용자가 변경할 수 있으므로 권한 판정에 사용하지 않는다.

5. 테이블 계약

5.1 public.recipes

- **id** text primary key
- **name** text not null
- **category** text not null
- **sub_category** text not null
- **chosung** text null
- **status** public.content_status not null default 'draft'
- **sort_order** integer not null default 0
- **created_at** timestampz not null default now()
- **updated_at** timestampz not null default now()

content_status 값은 draft, published, archived다.

카테고리 값:

- 음료
- 베이커리
- 이벤트

세부 카테고리 값:

- 음료: 카페인, 디카페인, 논커피, 에이드/티, 플랫치노, 생과일, 시즌 음료
- 베이커리: 베이글, 브레드, 케이크, 디저트
- 이벤트: 시즌, 세트, 페어링

현재 DB draft에는 category/sub_category CHECK 제약이 없다. Web UI와 API 계층에서 위 값을 검증하고, 추후 DB CHECK 제약 추가를 권장한다.

5.2 public.recipe_details

- `recipe_id` text primary key references public.recipes(id) on delete cascade
- `hero_visuals` jsonb not null default []
- `steps` jsonb not null default []
- `store_serving` jsonb not null default []
- `packaging` jsonb not null default []
- `delivery` jsonb not null default []
- `created_at` timestamptz not null default now()
- `updated_at` timestamptz not null default now()

`recipe_id`는 PK이므로 레시피마다 상세 row는 최대 1개다. `recipes`가 영구 삭제되면 상세 row도 cascade 삭제된다.

5.3 public.find_entries

- `id` text primary key
- `recipe_id` text not null
- `kind` public.find_entry_kind not null (material, pos)
- `title`, `summary`, `notes`, `chosung`
- `status`, `sort_order`, `payload` jsonb
- `created_at`, `updated_at`

현재 `recipe_id`에는 FK가 없다. `material`은 빈 문자열을 쓰고, `pos`만 recipe ID를 참조한다. 이 구조 때문에 레시피 영구 삭제 시 고아 참조가 생길 수 있다.

권장 개선:

- `recipe_id`를 nullable로 변경
- `material`이면 `recipe_id` is null
- `pos`이면 `recipe_id` is not null
- `recipes(id)` FK를 on delete restrict로 연결
- 영구 삭제 전에 POS 항목을 재연결 또는 archive

5.4 public.find_entry_keywords

- `entry_id` text references public.find_entries(id) on delete cascade
- `keyword` text not null
- `sort_order` integer not null default 0
- PK: (`entry_id`, `keyword`)

6. 레시피 상세 JSON 계약

DB에는 로컬 URI나 public URL이 아니라 `storagePath`만 저장한다.

```
{
  "hero_visuals": [
    {
      "id": "visual-main",
      "title": "완성 이미지",
      "desc": "HOT 기준",
      "storagePath": "recipes/recipe-id/visual-main/unique-name.jpg"
    }
  ],
  "steps": [
    {
      "id": "step-1",
      "title": "제조 순서",
      "details": ["샷을 추출한다", "재료를 혼합한다"],
      "visuals": []
    }
  ],
  "store_serving": [],
  "packaging": [],
  "delivery": []
}
```

Visual 필드:

- 필수: **id**, **title**
- 선택: **desc**, **description**, **storagePath**
- 저장 금지: **image**, **imageUri** (앱 런타임 전용)
- 새 Web 서비스에서는 설명 필드를 **desc**로 통일하는 것이 현재 모바일 저장 형식과 가장 잘 맞는다.

Step 필드:

- **id**: string
- **title**: string
- **details**: string[]
- **visuals**: RecipeVisual[]

7. 초성(chosung) 생성 계약

모바일 앱은 메뉴명을 저장할 때 한글 음절을 초성으로 변환한 뒤 모든 공백을 제거한다.

```
const CHOSUNG = [
  "ㄱ", "ㄴ", "ㄷ", "ㄹ", "ㅁ", "ㅂ", "ㅅ", "ㅇ",
  "ㅈ", "ㅊ", "ㅋ", "ㆁ", "ㄷ", "ㄹ", "ㅁ", "ㅂ", "ㅅ", "ㅇ",
  "ㅈ", "ㅊ", "ㅋ", "ㆁ", "ㄷ", "ㄹ", "ㅁ", "ㅂ", "ㅅ", "ㅇ",
] as const;

function getChosung(text: string) {
  return [...text].map(char => {
    const code = char.charCodeAt(0) - 0xac00;
    return code < 0 || code > 11171 ? char : CHOSUNG[Math.floor(code / 588)];
  }).join("").replace(/Ws/g, "");
}
```

Web에서 이름을 수정할 때 **chosung**도 반드시 함께 갱신한다.

8. 조회 구현 예시

```
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
);
```

게시된 레시피 목록과 상세:

```
const { data, error } = await supabase
  .from("recipes")
  .select("*, recipe_details(*)")
  .eq("status", "published")
  .order("sort_order", { ascending: true })
  .order("name", { ascending: true });
```

```
if (error) throw error;
```

관리자 목록(archive 제외):

```
const { data, error } = await supabase
  .from("recipes")
  .select("*, recipe_details(*)")
  .neq("status", "archived")
  .order("sort_order", { ascending: true });
```

```
if (error) throw error;
```

단일 레시피:

```
const { data, error } = await supabase
  .from("recipes")
  .select("*, recipe_details(*)")
  .eq("id", recipeId)
  .single();
```

```
if (error) throw error;
```

이미지 public URL:

```
const url = supabase.storage
  .from("recipe-content")
  .getPublicUrl(storagePath)
  .data.publicUrl;
```

9. 이미지 업로드 계약

버킷: **recipe-content**

허용 형식:

- **image/jpeg**
- **image/png**
- **image/webp**
- 최대 6 MiB

경로:

```
recipes/{recipe_id}/{visual_id}/{unique_file_name}
```

권장 업로드:

```
const path = [
  "recipes",
  sanitize(recipeId),
  sanitize(imageId),
  `${Date.now()}-${crypto.randomUUID()}.${extension}`,
].join("/");
```

```
const { error } = await supabase.storage
  .from("recipe-content")
  .upload(path, file, {
    cacheControl: "31536000",
    contentType: file.type,
    upsert: false,
  });
```

```
if (error) throw error;
```

기존 object path를 덮어쓰지 않는다. 새 파일을 고유 경로에 업로드한 뒤 DB 저장 성공 후 이전 경로를 제거한다.

안전한 순서:

1. 파일 확장자, MIME, 6 MiB 제한 검증
2. 새 이미지를 고유 path로 업로드
3. DB의 레시피와 상세 저장
4. DB 저장 성공 후 교체된 이전 object path 제거
5. DB 저장 실패 시 이번 요청에서 새로 올린 object를 best-effort 정리

Storage 삭제는 SQL로 [storage.objects](#)를 직접 수정하지 말고 Storage API의 [.remove\(paths\)](#)를 사용한다.

10. 등록 및 수정

10.1 현재 앱과 호환되는 직접 upsert

```
const now = new Date().toISOString();
const recipeRow = {
  id: recipeld,
  name: name.trim(),
  category,
  sub_category: subCategory,
  chosung: getChosung(name),
  status,
  sort_order: sortOrder,
  created_at: existingCreatedAt ?? now,
  updated_at: now,
};

const detailRow = {
  recipe_id: recipeld,
  hero_visuals: detail.heroVisuals,
  steps: detail.steps,
  store_serving: detail.storeServing,
  packaging: detail.packaging,
  delivery: detail.delivery,
  created_at: existingDetailCreatedAt ?? now,
  updated_at: now,
};

const recipeResult = await supabase
  .from("recipes")
  .upsert(recipeRow, { onConflict: "id" });
if (recipeResult.error) throw recipeResult.error;

const detailResult = await supabase
  .from("recipe_details")
  .upsert(detailRow, { onConflict: "recipe_id" });
if (detailResult.error) throw detailResult.error;
```

이 방식은 현재 Expo 앱과 동일하지만 두 Data API 요청 사이에 원자성이 없다. 첫 요청 성공 후 두 번째 요청이 실패하면 마스터만 저장될 수 있다.

10.2 Web 서비스 권장 방식

- `upsert_recipe_bundle(...)` 같은 `security invoker` RPC를 만든다.
- 함수 한 트랜잭션 안에서 recipe와 detail을 함께 검증 및 upsert한다.
- 수정 시 기존 `updated_at`을 인자로 받아 optimistic concurrency를 적용한다.
- 이미 다른 사용자가 수정했다면 409 성격의 충돌 오류를 반환하고 덮어쓰지 않는다.
- 이미지 업로드는 DB 트랜잭션 밖에서 처리하되, 새 object와 이전 object 목록을 명시적으로 관리한다.

권장 입력:

```
type UpsertRecipeBundleInput = {
  recipeId: string;
  name: string;
  category: "음료" | "베이커리" | "이벤트";
  subCategory: string;
  status: "draft" | "published" | "archived";
  sortOrder: number;
  expectedUpdatedAt?: string;
  detail: RecipeDetail;
};
```

신규 ID는 timestamp 단독보다 충돌 가능성이 낮은 `recipe-${crypto.randomUUID()}` 형식을 권장한다.

11. 게시 상태 관리

- 신규 작성 중: **draft**
- 직원에게 배포할 준비 완료: **published**
- 목록과 다음 앱 동기화에서 제외: **archived**

게시:

```
await supabase
  .from("recipes")
  .update({ status: "published", updated_at: new Date().toISOString() })
  .eq("id", recipeId);
```

기본 삭제(권장 archive):

```
await supabase
  .from("recipes")
  .update({ status: "archived", updated_at: new Date().toISOString() })
  .eq("id", recipeId);
```

복구:

```
await supabase
  .from("recipes")
  .update({ status: "draft", updated_at: new Date().toISOString() })
  .eq("id", recipeId);
```

archive는 detail과 이미지 path를 유지하므로 복구가 쉽다. 직원 앱은 다음 콘텐츠 동기화부터 archived 레시피를 제외한다.

12. 영구 삭제 절차

현재 모바일 앱의 삭제는 다음 순서다.

1. **recipe_details**를 읽어 이미지 path 수집
2. **recipes**를 실제 DELETE
3. FK cascade로 **recipe_details** 삭제
4. Storage API로 이미지 best-effort 제거

새 Web 서비스에서는 영구 삭제를 일반 삭제 버튼과 분리한다.

영구 삭제 전 필수 검사:

```
const { data: references, error } = await supabase
  .from("find_entries")
  .select("id, title, status")
  .eq("kind", "pos")
  .eq("recipe_id", recipeld)
  .neq("status", "archived");

if (error) throw error;
if (references.length > 0) {
  throw new Error("연결된 POS 통합검색 항목을 먼저 재연결하거나 archive해야 합니다.");
}
```

삭제:

```
const { error } = await supabase
  .from("recipes")
  .delete()
  .eq("id", recipeld);

if (error) throw error;
```

DB 삭제와 Storage 삭제는 하나의 Postgres 트랜잭션으로 묶을 수 없다. 서버 작업으로 실행하고, 실패한 Storage 정리를 재 시도할 수 있는 로그 또는 cleanup queue를 두는 것이 안전하다.

13. 보안 모델

현재 호환 모델

현재 저장소 SQL은 **anon**, **authenticated**에 source 테이블 SELECT/INSERT/UPDATE/DELETE를 주고 RLS 조건도 **using (true), with check (true)**다. 앱의 로컬 사장님 비밀번호는 DB가 확인하지 않는다.

이 모델은 매장 전용 trusted client라는 제한된 가정에만 맞는다. 공개 Web에 그대로 적용하면 publishable key를 가진 누구나 레시피를 변경할 수 있다.

Web 서비스 권장 모델

1. Supabase Auth 로그인 적용
2. 관리자 계정의 **app_metadata.app_role = 'sajang'**
3. anon은 published SELECT만 허용하거나 source 테이블 접근 제거
4. authenticated도 role claim을 만족할 때만 INSERT/UPDATE/DELETE 허용
5. 브라우저는 publishable key와 사용자 session만 사용
6. service role은 필요할 때만 서버 secret으로 사용
7. 모든 허용·거부 케이스를 RLS 테스트로 자동 검증

주의: RLS를 강화하면 현재 anon 쓰기에 의존하는 모바일 사장님 화면이 함께 막힌다. Web만 먼저 강화하지 말고 모바일 사장님 인증 전환 계획과 같이 진행해야 한다.

14. 현재 스키마의 개선 권고

우선순위 높음:

- Web 공개 전에 anon write 제거 및 Supabase Auth 도입
- recipe bundle 저장용 원자적 **security invoker** RPC

- `find_entries.recipe_id` nullable + POS 전용 FK + CHECK 제약
- 현재 발견된 POS 고아 참조 1건 보정
- 기본 삭제를 hard delete에서 archive로 변경
- recipe/detail `updated_at` 자동 갱신 trigger 또는 RPC 내 강제 갱신
- category/sub_category/status 조합 검증 제약

우선순위 중간:

- 수정 충돌 방지를 위한 optimistic concurrency
- 영구 삭제 및 Storage cleanup 감사 로그
- `recipes(status, category, sub_category, sort_order)` 인덱스 유지
- `find_entries(recipe_id)` 인덱스 유지
- Storage의 `recipes/`, `find/` 경로에 대한 실제 RLS 정책을 관리자 SQL로 점검하고 migration 파일에 기록

15. Web 구현 체크리스트

- [] 로그인하지 않은 사용자는 관리자 화면에 접근할 수 없다.
- [] publishable key만 브라우저에 포함된다.
- [] 서비스 역할 키가 Git, 클라이언트, 로그에 없다.
- [] 이름·카테고리·세부 카테고리·status를 검증한다.
- [] 이름 변경 시 chosung도 갱신한다.
- [] JSON에는 storagePath만 저장하고 image/imageUri는 제거한다.
- [] 이미지 크기와 MIME을 업로드 전에 검사한다.
- [] recipe와 detail 저장이 원자적으로 처리된다.
- [] 수정 충돌을 감지한다.
- [] 일반 삭제는 archive로 처리한다.
- [] 영구 삭제 전 POS 참조를 검사한다.
- [] 교체/삭제 이미지 cleanup 실패를 재시도할 수 있다.
- [] 저장 후 앱 콘텐츠 동기화가 필요함을 UI에 안내한다.
- [] RLS allow/deny 테스트와 실제 publishable-key 통합 테스트를 실행한다.

16. 다른 AI에게 전달할 시작 프롬프트

첨부한 "EDIYA_MEJANG Supabase 레시피 관리 Web 서비스 인계서"를 source of truth로 읽어라.

목표:

- 기존 Supabase 프로젝트 nbeupbzteeaeuznuaibm의 레시피를 관리하는 Web 관리자 서비스를 구현한다.
- recipes + recipe_details를 묶어 조회, 등록, 수정한다.
- 상태는 draft/published/archived를 사용하고 일반 삭제는 archived로 처리한다.
- 영구 삭제는 POS 통합검색 참조 검사와 Storage cleanup을 포함한 별도 관리자 동작으로 제한한다.
- recipe-content 버킷에는 고유 object path로 이미지를 업로드하고 DB JSON에는 storagePath만 저장한다.
- 메뉴명 변경 시 모바일 앱과 동일한 chosung 값을 갱신한다.

보안:

- 브라우저에는 Supabase publishable key만 사용한다.
- service_role/secret key를 클라이언트에 노출하지 않는다.
- 현재 anon write 정책은 공개 Web에 부적합하므로 Supabase Auth와 app_metadata 기반 sajang role RLS 전환 계획을 먼저 제안한다.
- 모바일 사장님 화면이 현재 anon write에 의존하므로 호환성 영향을 명시한다.

구현 전에:

1. 프레임워크와 인증 방식을 확인한다.
2. 실제 Supabase 스키마, RLS, Storage policy를 관리자 권한으로 읽기 전용 검사한다.
3. POS 고아 recipe 참조 1건의 보정 계획을 제시한다.
4. 원자적 recipe bundle RPC와 migration 초안을 제시하고 승인 후 적용한다.
5. 등록/수정/archive/복구/영구삭제/RLS 거부 케이스를 테스트한다.

17. 근거 파일과 공식 참고자료

프로젝트 근거:

- [supabase/schema/content_source_schema.sql](#)
- [supabase/schema/sajang_source_public_access.sql](#)
- [src/database/content-source/content-source.type.ts](#)
- [src/database/recipe/recipe.type.ts](#)
- [src/database/recipe/recipe-details.type.ts](#)
- [src/lib/sajang-content/supabase-content-repository.ts](#)
- [src/lib/sajang-content/supabase-content-images.ts](#)
- [src/components/features/sajang/menu-management/recipe-editor-form.tsx](#)
- [src/lib/content-cache/sync-recipe-search-cache.ts](#)
- [src/lib/content-cache/recipe-search-cache.ts](#)

Supabase 공식 참고자료:

- RLS: <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Data API 보안: <https://supabase.com/docs/guides/api/securing-your-api>
- Storage 접근제어: <https://supabase.com/docs/guides/storage/security/access-control>
- Storage 버킷: <https://supabase.com/docs/guides/storage/buckets/fundamentals>
- 2026 Data API 명시적 GRANT 변경: <https://supabase.com/changelog/45329-breaking-change-tables-not-exposed-to-data-and-graphql-api-automatically>